Воспроизведение результатов статьи

CODA: Rewriting Transformer Blocks as
GEMM-Epilogue Programs

Михаил Филимонов

Реализация

Описание сетапа в результатах и то, что написано синем в главах 2 и 3, (оно и только оно) было написано с помощью ChatGPT.

Содержание

1 Результаты **2**

1.1 Условия сравнения 2

1.2 Скорость 2

1.3 Корректность и ограничения 2

2 Мой путь **5**

2.1 Чтение статьи и выбор инструментария 5

2.2 Forward для GEMM + residual + RMSNorm + GEMM 5

2.3 Backward и проверка корректности 5

2.4 GEMM + pairwise: RoPE 6

2.5 GEMM + cross-entropy 6

2.6 Первый бенчмаркинг и срочные исправления 7

2.7 Типы данных и численная точность 7

3 Выводы **7**

1. Результаты

1.1. Условия сравнения

Прогон выполнен на NVIDIA H100 80GB HBM3 в BF16: PyTorch 2.10.0, CuTeDSL 4.4.2, Liger 0.8.0, FlashInfer 0.6.10.post1 и QuACK 0.4.1. Измерялись локальные блоки, а не полные Transformer-модели; поэтому это частичное воспроизведение идеи CODA, а не численных результатов статьи. «Моя CODA» ниже — реализация проекта на Triton, не оригинальные ядра авторов.

Для RMS использованы 16384 токена и проекции $d \to d \to 3d$; для RoPE — $M = d = 4096/8192$ и $d \to 3d \to d$; для CE — $M = d = 4096/8192$ и словарь 32768, loss без усреднения. Бейзлайн — наивная композиция PyTorch с `torch.compile`; название cuBLAS в CSV обозначает PyTorch с библиотечным GEMM. Остальные варианты компилировались, когда это поддерживалось, иначе измерялись напрямую.

После 10 прогревов выполнялись 30 замеров CUDA-событиями. Генерация входов, компиляция и autotuning исключены из времени; в backward подготовка графа также вне замера. Выходы и все градиенты проверялись с общим upstream gradient: для BF16 atol = rtol = 10^{-2}, для FP32 — 10^{-5} и 10^{-4}. Источник таблиц и рисунка — `results/bench_20261010_192137.csv`.

1.2. Скорость

Отрицательный результат. Ускорение своего решения относительно compiled PyTorch не воспроизведено для RMS forward, RoPE и CE forward+backward. В этих режимах оно медленнее в 1,3–3,3 раза. Особенно проигрывает CE forward: speedup около 0,31×. Liger, FlashInfer и QuACK в большинстве случаев близки к compiled-бейзлайну; малые отличия около единицы не следует трактовать как устойчивое преимущество.

Положительный результат. CE backward даёт 1,09× при $d = 4096$ и 1,04× при $d = 8192$. Однако при большем размере разброс времён сопоставим с выигрышем, а полный проход остаётся медленнее бейзлайна. Относительно eager PyTorch свой CE forward+backward быстрее в 6,8 и 2,2 раза, но compiled PyTorch ускоряет тот же эталон ещё сильнее.

1.3. Корректность и ограничения

Во всей сетке, включая малые проверки, получены 183 успешных замера, 39 неподдерживаемых вариантов и 24 провала проверки градиентов; ошибок запуска нет. Все поддерживаемые малые проверки прошли, включая RMS с несколькими батчами, хвостами тайлов и ненулевым ε.

Основная незакрытая проблема — RMS backward в BF16 на целевых размерах. Для всех трёх d в B и FB проверку не прошли моя реализация, compiled PyTorch, Liger и QuACK. Различия затрагивают небольшую долю элементов, но превышают заданный допуск; причина ещё не установлена. Допуски не повышались, времена этих вариантов не публикуются. Поскольку compiled-бейзлайн тоже не прошёл проверку, speedup в этих панелях отсутствует. FlashInfer измерен только в forward; Liger fused CE исключён из B/FB из-за ограничения адаптера для произвольного per-token upstream gradient.

Таким образом, корректность подтверждена для целевых forward-прогонов и поддерживаемых RoPE/CE backward, но ускорение полного блока своим решением не показано. Выигрыш CE backward — локальный положительный результат, который пока не компенсирует стоимость forward.

Таблица 1: Целевые времена своей реализации и compiled PyTorch, мс: среднее ± стандартное отклонение. F — forward, B — backward, FB — forward+backward; «—» означает провал проверки корректности.

Блок	Режим	d	Compiled PyTorch	Моя CODA	Speedup
RMS	F	2048	0.85 ± 0.01	1.57 ± 0.02	0.54×
RMS	B	2048	—	—	—
RMS	FB	2048	—	—	—
RMS	F	4096	3.38 ± 0.05	5.18 ± 0.03	0.65×
RMS	B	4096	—	—	—
RMS	FB	4096	—	—	—
RMS	F	8192	13.20 ± 0.09	25.07 ± 1.24	0.53×
RMS	B	8192	—	—	—
RMS	FB	8192	—	—	—
RoPE	F	4096	1.24 ± 0.04	2.67 ± 0.02	0.46×
RoPE	B	4096	2.62 ± 0.10	3.69 ± 0.04	0.71×
RoPE	FB	4096	3.92 ± 0.03	6.45 ± 0.04	0.61×
RoPE	F	8192	10.08 ± 0.06	21.16 ± 1.08	0.48×
RoPE	B	8192	20.28 ± 0.76	26.69 ± 1.25	0.76×
RoPE	FB	8192	30.61 ± 0.60	47.88 ± 0.81	0.64×
CE	F	4096	1.75 ± 0.08	5.68 ± 0.06	0.31×
CE	B	4096	3.82 ± 0.08	3.52 ± 0.06	1.09×
CE	FB	4096	5.51 ± 0.07	9.04 ± 0.04	0.61×
CE	F	8192	6.90 ± 0.09	22.56 ± 0.06	0.31×
CE	B	8192	13.84 ± 0.34	13.26 ± 0.70	1.04×
CE	FB	8192	20.41 ± 0.58	35.69 ± 0.63	0.57×

Таблица 2: Speedup относительно compiled PyTorch (1,00×). «П» — проверка градиентов не пройдена; «Н» — режим не поддержан адаптером. Compiled PyTorch для RMS B/FB сам не прошёл проверку.

Блок	Режим	d	Eager	Моя CODA	Liger	FlashInfer	QuACK	Liger fused
RMS	F	2048	0.76	0.54	0.94	0.99	0.93	—
RMS	B	2048	—	П	П	Н	П	—
RMS	FB	2048	—	П	П	Н	П	—
RMS	F	4096	0.90	0.65	0.99	1.00	1.01	—
RMS	B	4096	—	П	П	Н	П	—
RMS	FB	4096	—	П	П	Н	П	—
RMS	F	8192	0.93	0.53	0.99	0.99	1.02	—
RMS	B	8192	—	П	П	Н	П	—
RMS	FB	8192	—	П	П	Н	П	—
RoPE	F	4096	0.56	0.46	0.93	1.07	0.95	—
RoPE	B	4096	0.67	0.71	0.99	Н	0.96	—
RoPE	FB	4096	0.65	0.61	0.94	Н	0.94	—
RoPE	F	8192	0.76	0.48	0.98	1.01	1.01	—
RoPE	B	8192	0.81	0.76	0.99	Н	1.01	—
RoPE	FB	8192	0.79	0.64	0.98	Н	1.01	—
CE	F	4096	0.04	0.31	0.80	—	0.98	0.87
CE	B	4096	0.18	1.09	1.02	—	1.12	Н
CE	FB	4096	0.09	0.61	0.90	—	1.03	Н
CE	F	8192	0.15	0.31	0.88	—	1.03	0.97
CE	B	8192	0.45	1.04	0.96	—	1.06	Н
CE	FB	8192	0.26	0.57	0.89	—	1.02	Н

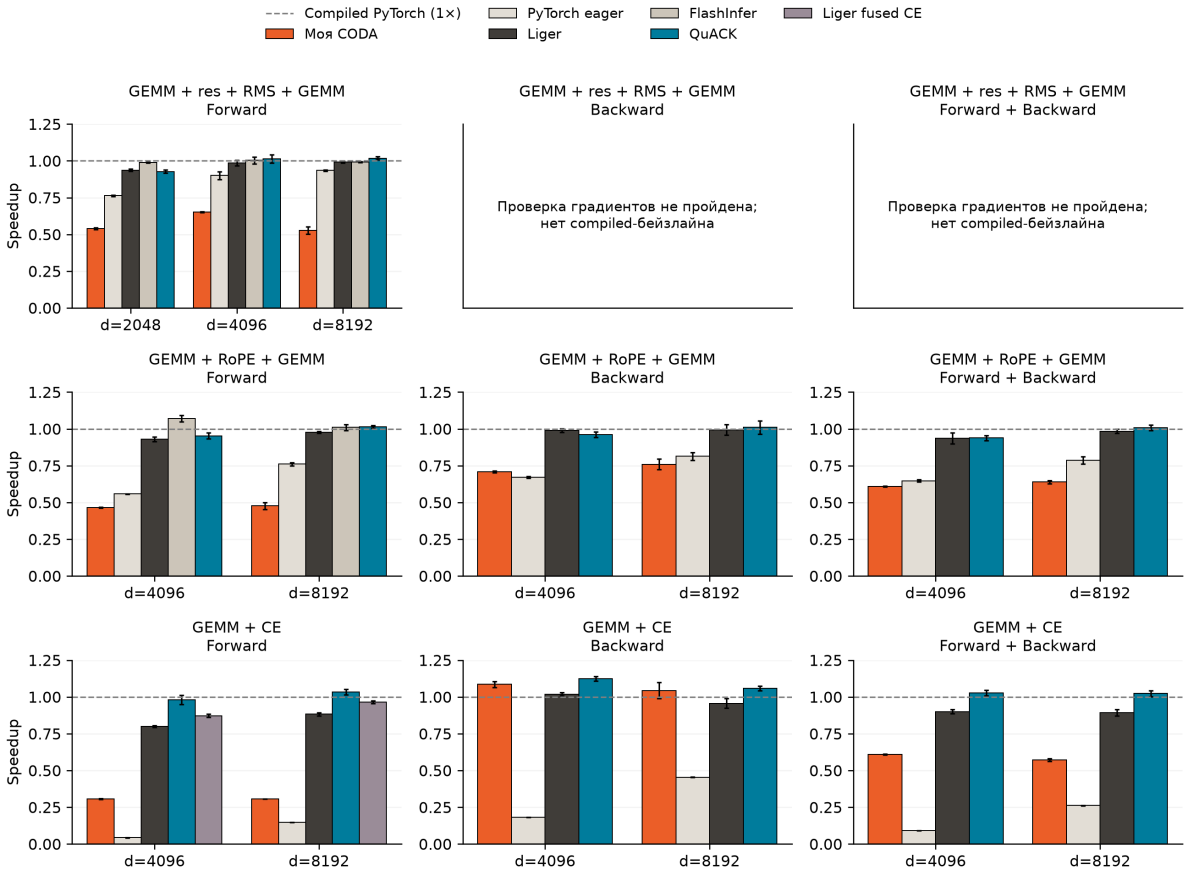


Рис. 1: Объединённое сравнение в формате сетки: строки — RMS, RoPE, CE; столбцы — F, B, FB. H100, BF16. Speedup = $t_{\text{compiled}}/t_{\text{variant}}$; выше пунктирной линии — быстрее compiled PyTorch. Штрихи показывают стандартное отклонение времени варианта, пересчитанное в speedup при фиксированном среднем времени бейзлайна; это не доверительные интервалы. Неподдерживаемые варианты отсутствуют, проваленные проверки не заменены нулевыми результатами.

2. Мой путь

2.1. Чтение статьи и выбор инструментария

Начал с того, что прочитал статью раза три с фокусом на `GEMM + res + RMSNorm + GEMM`, потому что планировал либо не реализовывать другие виды эпилогов, либо оставить их на потом. Идейно понял, как именно хотят поменять местами вычисления, но не до конца понимал, где именно это даст прибавку по скорости и в чём выгода, пока не начал реализовывать.

Авторы статьи реализовывали ядра на CuTeDSL. Я же, едва погрузившись на данный уровень взаимодействия с тензорами, решил остановиться на Triton, так как успел его потрогать и был семинар по нему. Полного повторения сетапа экспериментов не будет (и, насколько я понимаю, для `GEMM + pairwise` CuTeDSL предоставляет необходимый инструментарий для изменения логического расположения элементов в данных, что необходимо), но я хотя бы смогу реализовать это сам, без LLM-ки.

Реализованы локальные RMS-, RoPE- и CE-блоки с forward/backward; attention, embedding и полная Transformer-модель не воспроизводились. Итоговый прогон: H100 80GB HBM3, PyTorch 2.10.0 с CUDA 12.8, Triton 3.6.0. В pairwise-эпилоге соседние признаки должны попадать в одну пару: в Triton это `reshape` тайла в $(M_{\text{tile}}, K_{\text{tile}}/2, 2)$ и `split`. Смена DSL сама по себе не делает RoPE невозможным; отличие здесь в реализации layout и расписании GEMM.

2.2. Forward для GEMM + residual + RMSNorm + GEMM

Возможно, я сделал неправильно, но потратил на написание ядер процентов 90 времени, а оставшиеся 10 — на бенчмаркинг и исправление ошибок. Начал с GEMM, потом RMSNorm, потом соединил. Потом была неудачная попытка объединения `GEMM + res + RMSNorm + GEMM`, потому что я не понял, как именно получать partial RMS перед вторым GEMM, и попробовал реализовать через два ядра: GEMM и `res + RMSNorm + GEMM`. И хоть это и не было эпилогом таким, как это описано в статье, мне показалось, что так нормально и вроде бы этого авторы и хотели.

У меня получилось это завести и добиться того, что результаты получились одинаковыми, но, когда я пообщался с GPT о том, как и где оптимизировать, он сказал, что всё вообще неправильно: в статье написано по-другому и надо делать так-то и так-то. Осознал, что второе вспомогательное ядро всё-таки нужно и это нормально — писать с ним (так буквально и написано в статье, не знаю, каким боком я читал три раза). Переписал — получилось.

Получил удовольствие от дебагинга, потому что первая версия, которую я выдаю, больше похожа на разбитый пазл. Какие-то верные кусочки собраны, но надо доделывать. И процесс дебагинга — как сбор этого пазла. Особенно вместе с визуализатором очень приятно и не зубодробительно получается, мне понравилось. Эта стадия заняла процентов 30 времени, потому что у меня только настраивались мозги мыслить с точки зрения поинтеров и офсетов.

2.3. Backward и проверка корректности

Ещё процентов 20–30 времени писал backward сразу для всего блока, потому что никак не мог понять, откуда берутся формулы для backward в статье, что сохранять, а что пересчитывать, и так далее. Дошёл в итоге до схождения градиентов, но тоже не без ошибок.

Под «схождением» здесь понимается совпадение с autograd, а не сходимость обучения. Обозначим $H = XW_1 + Z$, $U = rH \odot \gamma$, $Y = UW_2$, $G = \nabla_Y L$ и $Q = GW_2^\top$. Для каждой строки

$$\widehat{s} = \frac{1}{d} \sum_j Q_j \gamma_j H_j r, \qquad D = \nabla_H L = r(Q \odot \gamma - rH\widehat{s}).$$

Тогда $\nabla_Z L = D$, $\nabla_X L = DW_1^\top$, $\nabla_{W_1} L = X_{\text{flat}}^\top D_{\text{flat}}$, $\nabla_{W_2} L = U_{\text{flat}}^\top G_{\text{flat}}$ и $\nabla_{\gamma_j} L = \sum_{b,i} Q_{bij} H_{bij} r_{bi}$. Здесь r уже включает ε; отдельного поправочного множителя к $\widehat{s}$ не требуется. Сохраняются $X, W_1, \gamma, W_2, r, H, U, Y$; Q вычисляется в backward. Исходная формула $\widehat{s} = d^{-1} \sum_k G_k Y_k$ математически правильна (раздел 3.2.4 и приложение A.2 статьи). В FP32 она сохранена. В BF16 используется свёртка через округлённый Q и восстановленное в FP32 $rH\gamma$, чтобы согласовать порядок округления с выбранным эталоном. Это численное изменение, а не исправление алгебры; на больших RMS-задачах проверки всё ещё не проходят.

При написании старался параллельно или изначально реализовывать сначала наивную версию, потом версию из статьи на PyTorch, потом уже версию из статьи на Triton, чтобы при дебаге было на что опираться.

2.4. GEMM + pairwise: RoPE

Ещё процентов 5–10 времени писал forward и backward для `GEMM + pairwise` на примере `GEMM + RoPE`, что оказалось несложным, так как backward почти такой же, а сама реализация не сильно отличается от дефолтной.

Под дефолтной понимается отдельное вращение после PyTorch GEMM. Для соседних признаков (a, b) и заранее подготовленных (c, s):

$$(a', b') = (ca - sb,\ sa + cb), \qquad (g_a, g_b) = (cg_{a'} + sg_{b'},\ -sg_{a'} + cg_{b'}).$$

В эпилоге `tl.split(tl.reshape(acc, (..., 2)))` выделяет пары; вычисляются `x1*cos-x2*sin` и `x1*sin+x2*cos`. Backward использует то же ядро с отрицательным `sin`. Таблицы имеют форму $(M, d/2)$, пары интерливированные; после RoPE добавлен второй GEMM. Проверяются выход и градиенты X, W_1, W_2; таблицы не обучаются.

2.5. GEMM + cross-entropy

Больше проблем возникло с `GEMM + CE`, потому что сначала в голове не укладывалось: «Я же пишу модель? Почему у меня здесь сразу лосс?» Потом всё уложилось (был просто день тупняка, видимо). Я подумал, что это похоже на FlashAttention из-за running softmax, который явно не описан в этой статье (потом оказалось, что я проглядел), но, мне кажется, тут нужен (нужен же ведь? GPT сказал, что нужен, и я согласен). Опыт написания FlashAttention у меня был, поэтому эта часть далась плюс-минус быстро.

Backward опять писал долго, потому что запутался с градиентами и тем, что пересчитывать (опирался на версию статьи). Потом мало что сошлось, и пришлось исправлять ошибки с GPT.

Loss возвращается отдельно для каждого токена, без среднего. Пусть g_i — его upstream gradient, $P_{ij} = e^{A_{ij} - \text{LSE}_i}$. Тогда

$$D_{ij} = g_i(P_{ij} - \mathbf{1}_{j=y_i}), \qquad \nabla_X L = DW^\top, \qquad \nabla_W L = X_{\text{flat}}^\top D_{\text{flat}}.$$

Сохраняются X, W, логиты, FP32 log-sum-exp и целочисленные метки; вероятности восстанавливаются в backward. Раньше логиты вычислялись повторным GEMM и создавался

плотный one-hot. Теперь используются сохранённые логиты и `scatter_add_` по меткам. Редукции и экспоненты выполняются в FP32; округление в dtype входа происходит перед GEMM градиентов. Для RoPE и CE устранено поэтапное BF16-округление градиентов весов по батчам: вместо суммы отдельных матричных произведений используется один GEMM по объединённым строкам.

2.6. Первый бенчмаркинг и срочные исправления

Первый бенчмаркинг показал, что мои версии ядер только замедляют вычисления, поэтому пришлось исправлять всё в быстром темпе с GPT (остаётся пара часов, хотя на всю домашку я дней 8–9 потратил). Начал фиксить с GPT и получил следующие ошибки.

Исправления относятся к корректности редукций, устройству буферов и лишней работе; точное происхождение каждой ранней ошибки по сохранившимся файлам установить нельзя. Для RMS выделение буферов привязано к устройству входа, masked loads дополнены нулями, upstream gradient приводится к contiguous, а число возвращаемых градиентов соответствует числу переданных аргументов. Редукция RMS обрабатывает ограниченное число строк; $\nabla\gamma$ сводится из отдельных FP32-частей без конкурирующих записей.

Лишние записи ненужного H_0, копии градиента residual и BF16-копия $W_2^\top$ убраны. В GEMM накопление изменено с `acc += tl.dot(a,b)` на `acc = tl.dot(a,b,acc)`. В CE последовательный цикл по словарю заменён параллельными тайлами с финальной редукцией. Подтверждённые изменения backward:

```
# раньше: P = x @ weight1; S = exp(P - lse)
probabilities = torch.exp(logits.float() - logsumexp[..., None])
# раньше: sum(x.transpose(2, 1) @ d_logits, dim=0)
grad_w = x.flatten(0, 1).T @ d_logits.flatten(0, 1)
```

2.7. Типы данных и численная точность

Ещё плохо разобрался, где какой тип данных надо ставить, и это тоже поботал с GPT. Изначально только аккумуляторы статстик делал в tl.float32 и не контролировал вход и выход моделей.

3. Выводы

Воспроизвести результаты не получилось. Прирост по скорости относительно compiled PyTorch есть только в GEMM + CE backward. Надеюсь, что это связано не только с моими ошибками, но и с тем, что в библиотеках GEMM уже гораздо лучше реализованы или я выбрал неудачные размеры тайлов.

В планах — разобраться вместе с GPT, какие у меня ещё есть ошибки, и исправить их. Повторить эксперименты после проверки размеров тайлов и результатов autotuning. Исправить backward там, где градиенты не совпадают с эталоном, и подробно разобраться, что было не так.

Несмотря на множество проблем, я всё-таки понял смысл статьи и паттерн GEMM + epilogue. Кстати, надо было реализовать его модульно, как в статье, а не переписывать GEMM каждый раз вручную. Хорошо потренировал Triton и навыки дебага, узнал, где какие типы данных применять.

Рекомендации GPT по поводу того, какие ошибки исправлять и что проверить:

1. Разобраться с несовпадением градиентов RMS backward на больших BF16-размерах. Проверку не проходят также compiled PyTorch, Liger и QuACK, поэтому нужно отдельно проверить порядок округлений и корректность эталона. Исходная формула $\widehat{s}$ математически верна; менять её без установленной причины не нужно.
2. Завершить преобразование RMS из статьи: перенести масштаб строки в эпилог второго GEMM и убрать отдельную запись нормированных активаций. Сейчас дополнительный проход по памяти сохраняется.
3. По профилю найти причину медленного GEMM и CE forward. Отдельно оценить стоимость матричных умножений, промежуточных буферов и запусков ядер; пока конкретный источник замедления не установлен.
4. Проверить выбранные autotuning-конфигурации для H100 на целевых формах. Неудачный размер тайла пока остаётся гипотезой; подбор должен завершаться до замера и сопровождаться повторной проверкой градиентов.